

Założenia i postęp prac nad aplikacją do zarządzania wydatkami

Michał Markuzel, Maksymilian Tulewicz, Antoni Piałucha, Jakub Wilczyński

28.04.2025

Spis treści

1	Wprowadzenie	3
2	Założenia ogólne	3
3	Funkcjonalności aplikacji	3
4	Technologie	3
5	Struktura bazy danych	4
6	Tworzenie tabel	5
7	Projekt aplikacji okienkowej	6
8	Procedury składowane (Stored Procedures)	8
8.1	Procedura <code>add_expense_to_group</code>	8
8.1.1	Funkcjonalność	9
8.1.2	Mechanizm działania	9
8.2	Procedura <code>add_equal_expense</code>	9
8.2.1	Funkcjonalność	11
8.2.2	Mechanizm działania	11
8.3	Procedura <code>create_group</code>	11
8.3.1	Funkcjonalność	12
8.3.2	Mechanizm działania	12
9	Triggery (Wyzwalacze)	12
9.1	Trigger <code>check_expense_completion</code>	12
9.1.1	Funkcjonalność	13
9.1.2	Mechanizm działania	13
9.2	Trigger <code>anonymize_user_data_after_delete</code>	13
9.2.1	Funkcjonalność	14
9.2.2	Mechanizm działania	14
10	Widoki (Views)	14
10.1	Widok <code>view_group_balances</code>	14
10.1.1	Funkcjonalność	15
10.2	Widok <code>view_group_expenses_details</code>	15
10.2.1	Funkcjonalność	15
10.3	Widok <code>view_user_groups</code>	16
10.3.1	Funkcjonalność	16
11	Ekran aplikacji	16
11.1	Logowanie do systemu	17
11.2	Lista grup użytkownika	18

1 Wprowadzenie

Aplikacja ma na celu umożliwienie grupom użytkowników wspólne zarządzanie wydatkami oraz rozliczanie kosztów między uczestnikami. Będzie inspirowana działaniem Tricounta, lecz dostosowana do specyficznych potrzeb użytkowników.

2 Założenia ogólne

- Aplikacja będzie dostępna w wersji desktopowej.
- Baza danych będzie przechowywać informacje o użytkownikach, grupach wydatków oraz transakcjach.
- System umożliwi automatyczne obliczanie udziałów w kosztach i proponowanie rozliczeń.
- Każdy użytkownik będzie miał możliwość dodawania wydatków i przypisywania ich do konkretnych osób w grupie.
- Historia wydatków będzie przechowywana w bazie danych i dostępna do przeglądania.

3 Funkcjonalności aplikacji

1. Autoryzacja i zarządzanie użytkownikami

- Rejestracja i logowanie użytkowników.
- Możliwość edycji profilu użytkownika.

2. Zarządzanie grupami wydatków

- Tworzenie i usuwanie grup.
- Dodawanie użytkowników do grupy.

3. Dodawanie i śledzenie wydatków

- Wprowadzanie wydatków z możliwością podziału na uczestników.
- Edycja i usuwanie wydatków.
- Przegląd historii transakcji.

4. Rozliczenia

- Automatyczne obliczanie kwot do rozliczenia.
- Możliwość oznaczania wydatków jako spłacone.

4 Technologie

- Język programowania: **Python**.
- GUI: **Qt/Terminal**.
- Baza danych: **MySQL**.
- System zarządzania wersjami: **Git**.

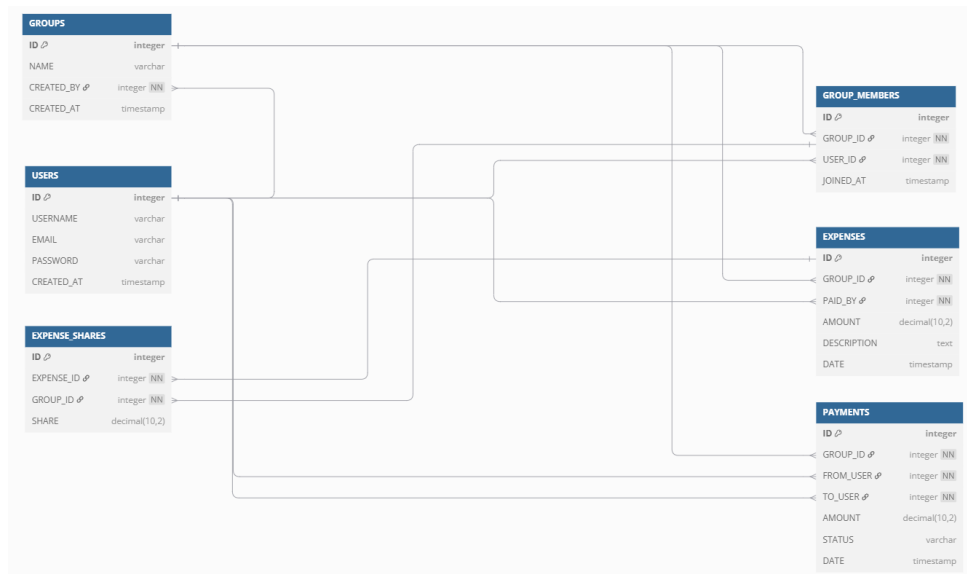
5 Struktura bazy danych

Baza danych będzie zawierała następujące tabele:

- **users** (id, username, email, password, created_at)
- **groups** (id, name, created_by, created_at)
- **group_members** (id, group_id, user_id, joined_at)
- **expenses** (id, group_id, paid_by, amount, description, date)
- **expense_shares** (id, expense_id, user_id, share)
- **payments** (id, group_id, from_user, to_user, amount, status, date)

Relacje między tabelami:

- Każda grupa (**groups**) jest tworzona przez użytkownika (**users**) poprzez klucz *created_by*.
- Członkowie grupy (**group_members**) należą do danej grupy (**groups**) i są powiązani z użytkownikami (**users**).
- Wydatki (**expenses**) są przypisane do grupy (**groups**), a każdy wydatek ma użytkownika, który go zapłacił (*paid_by* powiązane z **users**).
- Podział kosztów (**expense_shares**) wskazuje, jaka część wydatku przypada na konkretnego użytkownika.
- Płatności (**payments**) przechowują informacje o przelewach między użytkownikami w ramach grupy (**groups**).



Model relacyjny bazy danych.

6 Tworzenie tabel

```
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    hashed_password VARCHAR(255) NOT NULL,
    is_deleted BOOLEAN DEFAULT FALSE NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
);

CREATE TABLE group_table (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    created_by INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
);

CREATE TABLE group_members (
    id INT AUTO_INCREMENT PRIMARY KEY,
    group_id INT NOT NULL,
    user_id INT NOT NULL,
    joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
);

CREATE TABLE expenses (
    id INT AUTO_INCREMENT PRIMARY KEY,
    group_id INT NOT NULL,
    paid_by INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    description VARCHAR(255) NOT NULL,
    date TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
);
```

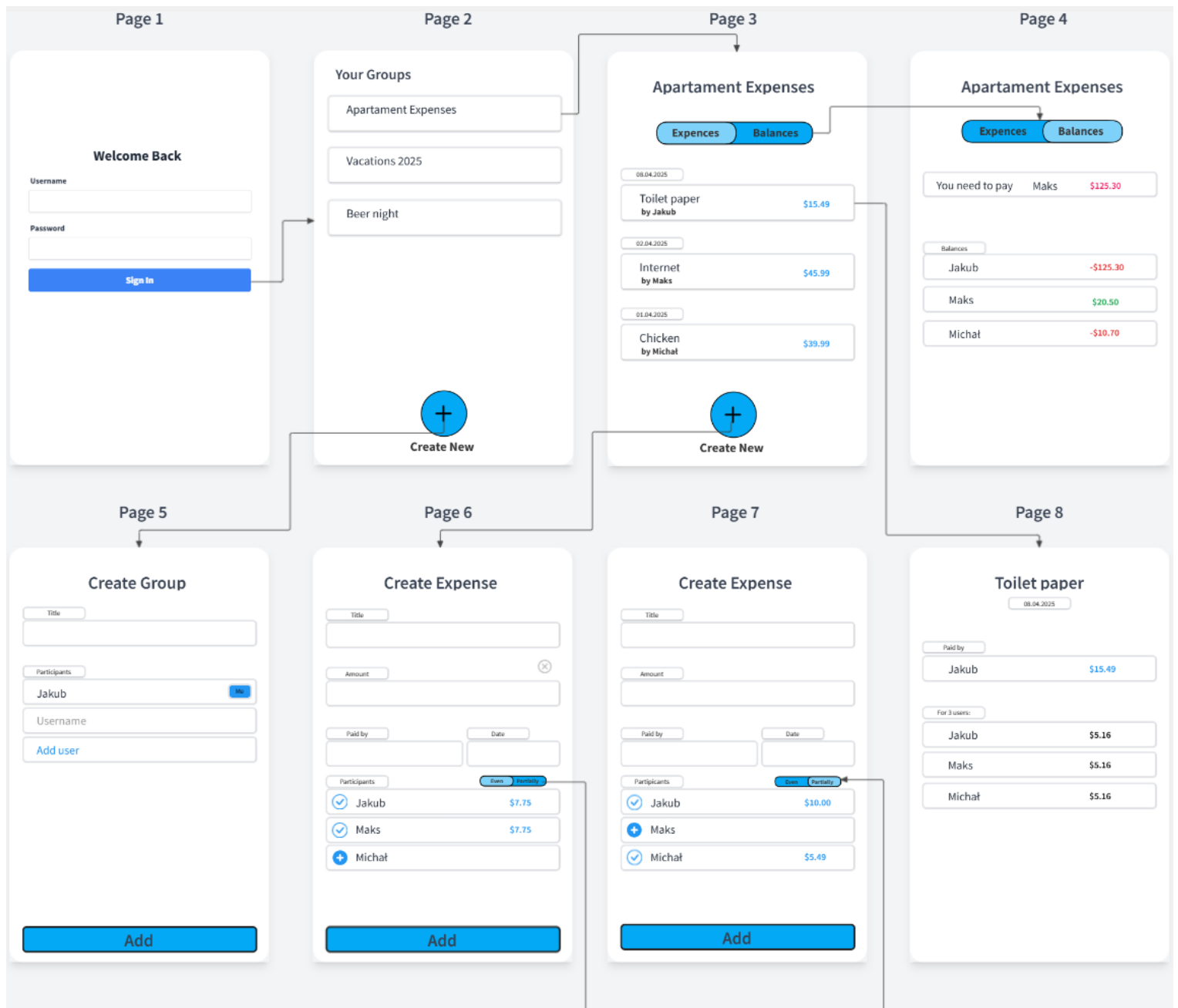
```
CREATE TABLE expense_participants (
    id INT AUTO_INCREMENT PRIMARY KEY,
    expense_id INT NOT NULL,
    user_id INT NOT NULL
);

CREATE TABLE expense_shares (
    id INT AUTO_INCREMENT PRIMARY KEY,
    expense_id INT NOT NULL,
    user_id INT NOT NULL,
    share DECIMAL(10,2) NOT NULL
);

CREATE TABLE payments (
    id INT AUTO_INCREMENT PRIMARY KEY,
    group_id INT NOT NULL,
    from_user INT NOT NULL,
    to_user INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    status ENUM('pending', 'completed', 'failed') NOT NULL DEFAULT 'pending',
    date TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
);
```

Utworzenie tabel w MySQL.

7 Projekt aplikacji okienkowej



1. Page1: Login

- Użytkownik loguje się.
- → Po zalogowaniu: **Page2**.

2. Page2: Lista grup użytkownika

- `SELECT * FROM view_user_groups WHERE user_id = ?`.
- Przycisk dla każdej grupy → **Page3** dla danej grupy.

3. Page3: Bilans rozliczeń w grupie

- `SELECT * FROM view_group_balances WHERE group_id = ?`.
- Przycisk "Expenses" → **Page4** (historia wydatków).
- Przycisk "Dodaj wydatek":
 - → Wariant 1: `CALL add_equal_expense(...)` – podział równy.
 - → Wariant 2: `CALL add_expense_to_group(...)` – ręczne udziały.

4. Page4: Szczegóły wydatków grupy

- `SELECT * FROM view_group_expenses_details WHERE group_id = ?`.

5. Page5: Profil użytkownika / Admin

- Przycisk "Usuń konto".
- `UPDATE users SET is_deleted = TRUE WHERE id = ?`.
- (Trigger anonimizuje dane użytkownika).

1. Page1: Login

- Użytkownik loguje się.
- → Po zalogowaniu: **Page2**.

2. Page2: Lista grup użytkownika

- `SELECT * FROM view_user_groups WHERE user_id = ?`.
- Przycisk dla każdej grupy → **Page3(group_id)**.

3. Page3: Szczegóły wydatków grupy

- `SELECT * FROM view_group_expenses_details WHERE group_id = ?`.
- Przycisk na dany wydatek → **Page8(expense_id)**.
- Przycisk na bilans rozliczeń w grupie → **group_id**.
- Przycisk "Dodaj wydatek":
 - Wariant A (równe udziały): `CALL add_equal_expense(group_id, payer_id, total_amount, expense_name)` **Page6**.
 - Wariant B (niestandardowe udziały): `CALL add_expense_to_group(group_id, payer_id, total_amount, expense_name, JSON_shares)` **Page7..**

4. Page4: Bilans rozliczeń w grupie

- `SELECT * FROM view_group_balances WHERE group_id = ?`.
- Przycisk "Expenses" → **Page3(group_id)**.

5. Page4: Bilans rozliczeń w grupie

- `CALL create_group(...)` **Page 6**

6. Page6, Page 7: Formularz dodawania wydatku

- Wysyła dane do jednej z procedur:
 - CALL add_equal_expense(...) Page 6
 - CALL add_expense_to_group(...) Page 7.
- Po zapisaniu: Page3(group_id).

7. Page8: Szczegóły konkretnego wydatku

- SELECT * FROM view_expense_participants WHERE expense_id = ?.
- Wyświetla: kto zapłacił, kwotę, udział każdego użytkownika.

8 Procedury składowane (Stored Procedures)

W ramach projektu zaimplementowano trzy główne procedury składowane, które obsługują podstawowe operacje na danych.

8.1 Procedura add_expense_to_group

Procedura umożliwia dodanie nowego wydatku do grupy wraz z przypisaniem niestandardowych udziałów dla uczestników.

```

1 DELIMITER //
2
3 CREATE PROCEDURE add_expense_to_group (
4     IN p_group_id INT,
5     IN p_paid_by INT,
6     IN p_amount DECIMAL(10,2),
7     IN p_description VARCHAR(255),
8     IN p_participants_shares TEXT -- np. '5:40,6:40,7:40'
9 )
10 BEGIN
11     DECLARE v_expense_id INT;
12     DECLARE v_pos INT DEFAULT 1;
13     DECLARE v_part TEXT;
14     DECLARE v_user_id INT;
15     DECLARE v_share DECIMAL(10,2);
16
17     -- 1. Dodaj wydatek
18     INSERT INTO expenses (group_id, paid_by, amount, description)
19     VALUES (p_group_id, p_paid_by, p_amount, p_description);
20
21     SET v_expense_id = LAST_INSERT_ID();
22
23     -- 2. Parsuj i dodawaj uczestnik w + udział y
24     WHILE LENGTH(p_participants_shares) > 0 DO
25         SET v_pos = LOCATE(',', p_participants_shares);
26
27         IF v_pos > 0 THEN
28             SET v_part = SUBSTRING(p_participants_shares, 1, v_pos -
29 1);
30             SET p_participants_shares = SUBSTRING(
31 p_participants_shares, v_pos + 1);
32         ELSE
33             SET v_part = p_participants_shares;
34             SET p_participants_shares = '';

```



```

33      END IF;
34
35      SET v_user_id = SUBSTRING_INDEX(v_part, ':', 1);
36      SET v_share = SUBSTRING_INDEX(v_part, ':', -1);
37
38      INSERT INTO expense_participants (expense_id, user_id)
39      VALUES (v_expense_id, v_user_id);
40
41      INSERT INTO expense_shares (expense_id, user_id, share)
42      VALUES (v_expense_id, v_user_id, v_share);
43  END WHILE;
44 END;
45 //
46
47 DELIMITER ;

```

Listing 1: Procedura add_expense_to_group

8.1.1 Funkcjonalność

- Rejestracja nowego wydatku w tabeli `expenses`
- Parsowanie listy uczestników i ich udziałów w wydatku
- Dodawanie rekordów do tabel `expense_participants` oraz `expense_shares`

8.1.2 Mechanizm działania

1. Tworzenie nowego rekordu wydatku
2. Iteracyjne przetwarzanie ciągu tekstowego z danymi uczestników
3. Dla każdego uczestnika, dodanie odpowiednich wpisów do powiązanych tabel

Procedura wykorzystuje mechanizm iteracyjnego przetwarzania tekstu z użyciem funkcji `SUBSTRING`, `SUBSTRING_INDEX` oraz operacji na łańcuchach znaków. Format wejściowy `p_participants_shares` to ciąg tekstowy w postaci `'id_użytkownika:udział, id_użytkownika:udział,...'`, co pozwala na elastyczne określanie udziałów dla poszczególnych uczestników.

8.2 Procedura add_equal_expense

Procedura służy do dodawania wydatków z równym podziałem kosztów pomiędzy uczestników.

```

1  DELIMITER //
2
3  CREATE PROCEDURE add_equal_expense (
4      IN p_group_id INT,
5      IN p_paid_by INT,
6      IN p_amount DECIMAL(10,2),
7      IN p_description VARCHAR(255),
8      IN p_participant_ids TEXT -- np. '5,6,7'
9  )
10 BEGIN

```

```

11 DECLARE v_expense_id INT;
12 DECLARE v_pos INT DEFAULT 1;
13 DECLARE v_id TEXT;
14 DECLARE v_user_id INT;
15 DECLARE v_count INT DEFAULT 0;
16 DECLARE v_share DECIMAL(10,2);
17 DECLARE temp_str TEXT;
18
19 -- policz ilu uczestnikow
20 SET temp_str = p_participant_ids;
21 WHILE LENGTH(temp_str) > 0 DO
22     SET v_pos = LOCATE(',', temp_str);
23     IF v_pos > 0 THEN
24         SET temp_str = SUBSTRING(temp_str, v_pos + 1);
25     ELSE
26         SET temp_str = '';
27     END IF;
28     SET v_count = v_count + 1;
29 END WHILE;
30
31 -- oblicz udzia
32 SET v_share = ROUND(p_amount / v_count, 2);
33
34 -- dodaj wydatek
35 INSERT INTO expenses (group_id, paid_by, amount, description)
36 VALUES (p_group_id, p_paid_by, p_amount, p_description);
37
38 SET v_expense_id = LAST_INSERT_ID();
39
40 -- dodaj uczestnika i udzialy
41 SET temp_str = p_participant_ids;
42 WHILE LENGTH(temp_str) > 0 DO
43     SET v_pos = LOCATE(',', temp_str);
44     IF v_pos > 0 THEN
45         SET v_id = SUBSTRING(temp_str, 1, v_pos - 1);
46         SET temp_str = SUBSTRING(temp_str, v_pos + 1);
47     ELSE
48         SET v_id = temp_str;
49         SET temp_str = '';
50     END IF;
51
52     SET v_user_id = CAST(v_id AS UNSIGNED);
53
54     INSERT INTO expense_participants (expense_id, user_id)
55     VALUES (v_expense_id, v_user_id);
56
57     INSERT INTO expense_shares (expense_id, user_id, share)
58     VALUES (v_expense_id, v_user_id, v_share);
59 END WHILE;
60 END;
61 //
62
63 DELIMITER ;

```

Listing 2: Procedura add_equal_expense

8.2.1 Funkcjonalność

- Automatyczne obliczanie równych udziałów dla wszystkich uczestników
- Rejestracja wydatku oraz przypisanie równych udziałów

8.2.2 Mechanizm działania

1. Zliczenie liczby uczestników przez parsowanie listy identyfikatorów
2. Obliczenie równego udziału dla każdej osoby ($p_amount / liczba_uczestników$)
3. Dodanie wydatku do systemu
4. Iteracyjne dodawanie uczestników i ich równych udziałów

Procedura znacząco upraszcza proces dodawania typowych wydatków, gdzie koszty są dzielone po równo. Format wejściowy `p_participant_ids` to ciąg tekstowy w postaci `'id_użytkownika,id_użytkownika,...'`, co pozwala na łatwe określenie uczestników wydatku.

8.3 Procedura `create_group`

Procedura umożliwia utworzenie nowej grupy rozliczeniowej wraz z przypisaniem jej członków.

```
1 DELIMITER //
```

```
2
```

```
3 CREATE PROCEDURE create_group (  
4     IN p_group_name VARCHAR(100),  
5     IN p_created_by INT,  
6     IN p_user_ids TEXT -- np. '2,3,4'  
7 )  
8 BEGIN  
9     DECLARE v_group_id INT;  
10    DECLARE v_pos INT DEFAULT 1;  
11    DECLARE v_user_id TEXT;  
12    DECLARE temp_str TEXT;  
13  
14    -- 1. Dodaj grup  
15    INSERT INTO group_table (name, created_by)  
16    VALUES (p_group_name, p_created_by);  
17  
18    SET v_group_id = LAST_INSERT_ID();  
19  
20    -- 2. Dodaj użytkowników do grupy  
21    SET temp_str = p_user_ids;  
22    WHILE LENGTH(temp_str) > 0 DO  
23        SET v_pos = LOCATE(',', temp_str);  
24        IF v_pos > 0 THEN  
25            SET v_user_id = SUBSTRING(temp_str, 1, v_pos - 1);  
26            SET temp_str = SUBSTRING(temp_str, v_pos + 1);  
27        ELSE  
28            SET v_user_id = temp_str;  
29            SET temp_str = '';  
30        END IF;
```

```

31
32      -- Dodaj do tabeli group_members
33      INSERT INTO group_members (group_id, user_id)
34      VALUES (v_group_id, CAST(v_user_id AS UNSIGNED));
35  END WHILE;
36 END;
37 //
38
39 DELIMITER ;

```

Listing 3: Procedura create_group

8.3.1 Funkcjonalność

- Tworzenie nowej grupy rozliczeniowej
- Automatyczne dodawanie członków grupy

8.3.2 Mechanizm działania

1. Utworzenie rekordu grupy w tabeli `group_table`
2. Iteracyjne przetwarzanie listy identyfikatorów użytkowników
3. Dodanie każdego użytkownika do tabeli `group_members` z powiązaniem do utworzonej grupy

Procedura ta usprawnia proces tworzenia grup i zapewnia spójność danych poprzez wykonanie wszystkich operacji w ramach jednej transakcji. Format wejściowy `p_user_ids` to ciąg tekstowy w postaci `'id_użytkownika,id_użytkownika,...'`, co pozwala na łatwe określenie członków grupy.

9 Triggery (Wyzwalacze)

W projekcie zaimplementowano dwa kluczowe triggery, które automatyzują pewne operacje na bazie danych:

9.1 Trigger `check_expense_completion`

Trigger uruchamiany po dodaniu nowej płatności, sprawdzający czy wszystkie wydatki w grupie zostały w pełni rozliczone.

```

1 DELIMITER //
2
3 CREATE TRIGGER check_expense_completion
4     AFTER INSERT
5     ON payments
6     FOR EACH ROW
7 BEGIN
8     DECLARE total_share DECIMAL(10, 2);
9     DECLARE total_paid DECIMAL(10, 2);
10
11     -- Oblicz sum udziału w w danym wydatku

```

```

32 SELECT SUM(share)
33 INTO total_share
34 FROM expense_shares es
35      JOIN expenses e ON es.expense_id = e.id
36 WHERE e.group_id = NEW.group_id;
37
38 -- Oblicz sum zapłaconych już płatności w grupie
39 SELECT SUM(amount)
40 INTO total_paid
41 FROM payments
42 WHERE group_id = NEW.group_id
43      AND status = 'completed';
44
45 -- Jeśli wyrównane, zaktualizuj wszystkie płatności jako
46 completed
47 IF total_share <= total_paid THEN
48     UPDATE payments
49     SET status = 'completed'
50     WHERE group_id = NEW.group_id;
51 END IF;
52 END;
53 //
54 DELIMITER ;

```

Listing 4: Trigger check_expense_completion

9.1.1 Funkcjonalność

- Automatyczne oznaczanie płatności jako zakończonych, gdy suma udziałów zostanie wyrównana
- Sprawdzanie stanu rozliczeń po każdej nowej płatności

9.1.2 Mechanizm działania

1. Obliczenie sumy wszystkich udziałów w wydatkach dla danej grupy
2. Obliczenie sumy zrealizowanych płatności w grupie
3. Jeśli sumy są sobie równe lub płatności przekraczają udziały, wszystkie płatności w grupie są oznaczane jako zakończone (**completed**)

Trigger ten znacząco zwiększa użyteczność systemu poprzez automatyczne śledzenie stanu rozliczeń. Gdy wszystkie należności zostają wyrównane, system automatycznie aktualizuje statusy płatności.

9.2 Trigger anonymize_user_data_after_delete

Trigger zapewniający ochronę danych osobowych przy usuwaniu konta użytkownika.

```

1 DELIMITER //
2
3 CREATE TRIGGER anonymize_user_data_after_delete
4 AFTER UPDATE ON users

```

```

5 FOR EACH ROW
6 BEGIN
7     IF NEW.is_deleted = TRUE AND OLD.is_deleted = FALSE THEN
8         -- Anonimizuj dane u ytkownika
9         UPDATE users
10        SET
11            username = CONCAT('anon_user_', NEW.id),
12            email = CONCAT('anon', NEW.id, '@example.com'),
13            hashed_password = 'deleted_user',
14            created_at = OLD.created_at -- zachowaj dat rejestracji
15        WHERE id = NEW.id;
16    END IF;
17 END;
18 //
19
20 DELIMITER ;

```

Listing 5: Trigger anonymize_user_data_after_delete

9.2.1 Funkcjonalność

- Anonimizacja danych użytkownika po oznaczeniu konta jako usunięte
- Zachowanie spójności bazy danych przy jednoczesnej ochronie danych osobowych

9.2.2 Mechanizm działania

1. Wykrywanie zmiany statusu `is_deleted` z `FALSE` na `TRUE`
2. Nadpisywanie danych osobowych użytkownika anonimowymi wartościami
3. Zachowanie niezbędnych powiązań z innymi tabelami przez utrzymanie identyfikatora użytkownika

Ten trigger implementuje zasadę minimalizacji danych zgodnie z wymogami ochrony danych osobowych, takimi jak RODO. Zamiast całkowicie usuwać dane użytkownika, co mogłoby naruszyć integralność referencyjną, trigger zastępuje dane osobowe pseudonimami.

10 Widoki (Views)

W projekcie zaimplementowano trzy widoki, które ułatwiają analizę i prezentację danych:

10.1 Widok `view_group_balances`

Widok prezentujący bieżący bilans rozliczeń pomiędzy użytkownikami w grupach.

```

1 CREATE OR REPLACE VIEW view_group_balances AS
2 SELECT g.id AS group_id,
3        u_from.username AS from_user,
4        u_to.username AS to_user,
5        SUM(p.amount) AS balance_due
6 FROM payments p
7 JOIN group_table g ON p.group_id = g.id

```

```

8      JOIN users u_from ON p.from_user = u_from.id
9      JOIN users u_to ON p.to_user = u_to.id
10 WHERE p.status != 'completed'
11 GROUP BY g.id, p.from_user, p.to_user;

```

Listing 6: Widok view_group_balances

10.1.1 Funkcjonalność

- Prezentacja sum niezakończonych płatności pomiędzy parami użytkowników
- Agregacja danych na poziomie grupy

Widok ten jest niezbędny do prezentacji aktualnych zobowiązań finansowych pomiędzy członkami grupy. Dzięki niemu można szybko uzyskać informację o tym, kto komu jest winien pieniądze w ramach danej grupy.

10.2 Widok view_group_expenses_details

Widok prezentujący szczegółowe informacje o wydatkach w grupach.

```

1 CREATE OR REPLACE VIEW view_group_expenses_details AS
2 SELECT g.id AS group_id,
3        g.name AS group_name,
4        e.id AS expense_id,
5        e.description,
6        e.amount,
7        u.username AS paid_by,
8        ep.user_id AS participant_id,
9        u2.username AS participant_name,
10       es.share
11 FROM expenses e
12      JOIN group_table g ON e.group_id = g.id
13      JOIN users u ON e.paid_by = u.id
14      JOIN expense_participants ep ON ep.expense_id = e.id
15      JOIN users u2 ON ep.user_id = u2.id
16      JOIN expense_shares es ON es.expense_id = e.id AND es.
       user_id = ep.user_id;

```

Listing 7: Widok view_group_expenses_details

10.2.1 Funkcjonalność

- Kompleksowy widok łączący dane z wielu tabel
- Prezentacja informacji o wydatkach wraz z danymi uczestników i ich udziałami

Widok ten dostarcza pełny obraz wydatków grupowych wraz ze wszystkimi istotnymi informacjami. Pozwala na łatwe wyświetlanie historii wydatków, udziałów poszczególnych uczestników oraz szczegółów dotyczących płatności.

10.3 Widok view_user_groups

Widok prezentujący grupy, do których należy użytkownik.

```
1 CREATE OR REPLACE VIEW view_user_groups AS
2 SELECT
3     gm.user_id,
4     u.username,
5     g.id AS group_id,
6     g.name AS group_name,
7     g.created_at AS group_created_at,
8     g.created_by
9 FROM group_members gm
10 JOIN group_table g ON gm.group_id = g.id
11 JOIN users u ON gm.user_id = u.id;
```

Listing 8: Widok view_user_groups

10.3.1 Funkcjonalność

- Szybki dostęp do informacji o przynależności użytkowników do grup
- Zwiększenie czytelności danych w aplikacji

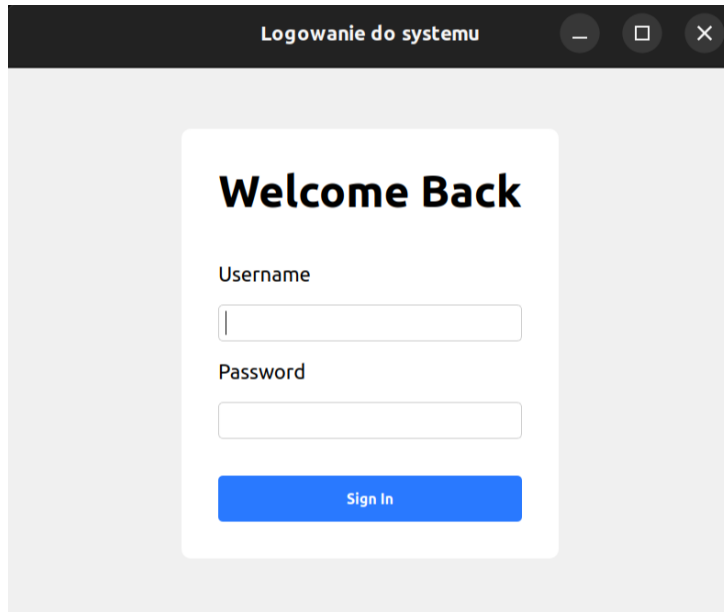
Widok ten upraszcza dostęp do informacji o grupach użytkownika, eliminując potrzebę skomplikowanych zapytań w kodzie aplikacji. Pozwala na szybkie pobranie listy grup, do których należy dany użytkownik, wraz z dodatkowymi informacjami o tych grupach.

11 Ekran aplikacji

Poniższe sprawozdanie przedstawiają wstępny widok aplikacji służącej do zarządzania wspólnymi wydatkami w grupach. Aplikacja jest w fazie projektowania i testowania podstawowej funkcjonalności interfejsu użytkownika. Na obecnym etapie dostępne są ekrany logowania oraz podgląd listy grup, do których należy zalogowany użytkownik. W przyszłości planowane są dalsze rozszerzenia funkcjonalności, takie jak zarządzanie wydatkami, dodawanie transakcji oraz rozliczanie kosztów między członkami grup.

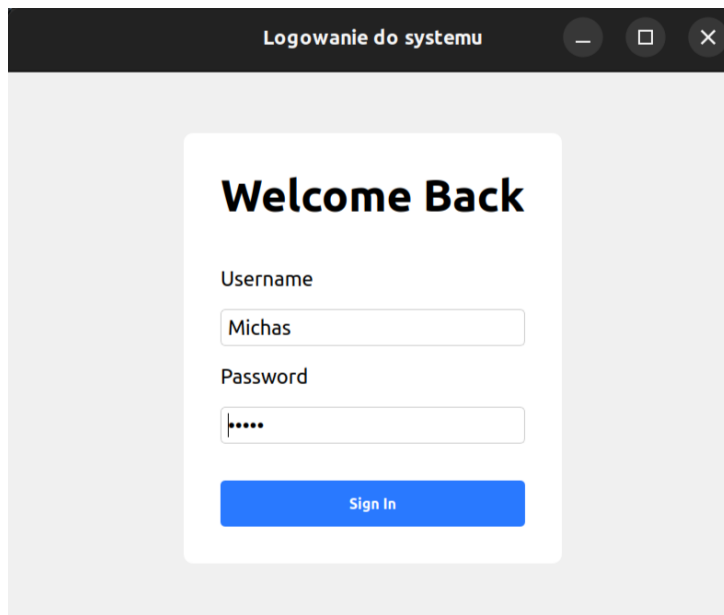
11.1 Logowanie do systemu

Na poniższych zrzutach ekranu przedstawiono ekran logowania do systemu:



The screenshot shows a web application window titled "Logowanie do systemu". Inside, there is a white card with the heading "Welcome Back". Below the heading are two input fields: "Username" and "Password". The "Username" field is empty, and the "Password" field is also empty. At the bottom of the card is a blue button labeled "Sign In".

Rysunek 1: Formularz logowania przed wpisaniem danych



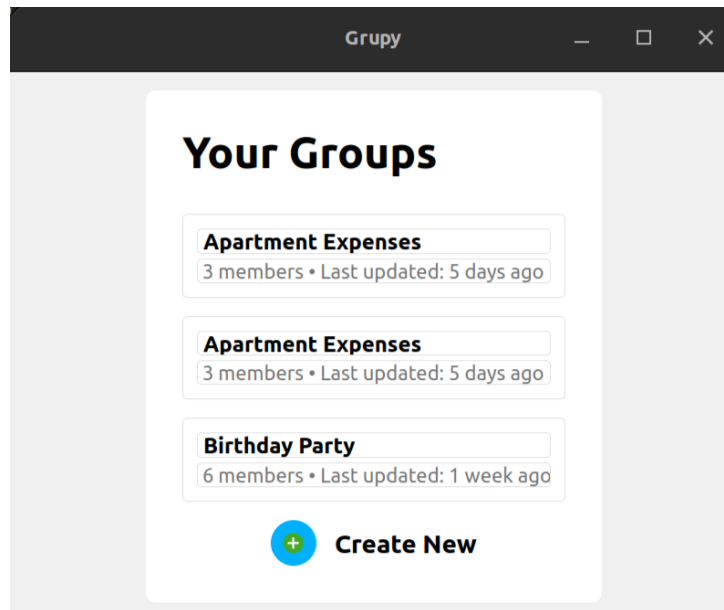
The screenshot shows the same web application window as before, but now the input fields are filled. The "Username" field contains the text "Michas", and the "Password" field contains five dots, indicating a masked password. The "Sign In" button remains at the bottom of the card.

Rysunek 2: Formularz logowania z wypełnionymi danymi

Użytkownik musi podać swoją nazwę użytkownika oraz hasło, aby uzyskać dostęp do aplikacji. Po pomyślnym zalogowaniu użytkownik zostaje przekierowany do ekranu głównego.

11.2 Lista grup użytkownika

Po zalogowaniu użytkownikowi wyświetlana jest lista jego grup:



Rysunek 3: Lista grup, do których należy użytkownik

Każda grupa zawiera nazwę, liczbę członków oraz datę ostatniej aktualizacji. W przykładzie widzimy dwie grupy o nazwie *Apartment Expenses* (wydatki związane z mieszkaniem) oraz jedną grupę *Birthday Party* (impreza urodzinowa). Użytkownik może również utworzyć nową grupę za pomocą przycisku **Create New**.